



Lista 4 — Estado, APIs e Roteamento

Aulas 31 a 40

Desenvolvimento Web

Exercícios sobre gerenciamento de estado, consumo de APIs REST, autenticação, roteamento com React Router e formulários avançados. Cenários de aplicações completas com backend.

Exercício 1

Cenário: App de clima (weather app) que consulta API externa e exibe previsão para 5 dias.

1. Use a API gratuita OpenWeatherMap (<https://openweathermap.org/api>) ou WeatherAPI para buscar clima da cidade digitada pelo usuário.
2. Crie componentes: SearchBar (input + botão), CurrentWeather (temperatura, ícone, umidade, vento), Forecast (lista dos próximos dias).
3. Gerencie estado com useReducer: {city, current, forecast, loading, error}. Dispatch ações: SEARCH, SUCCESS, ERROR.
4. Adicione tratamento de erros: cidade não encontrada, rede offline, limite de requisições excedido.
5. Extra: adicione geolocalização (navigator.geolocation) para clima automático ao carregar a página.

Exercício 2

Cenário: Aplicação precisa de autenticação com login, registro e proteção de rotas.

1. Crie um AuthContext que fornece { user, login, logout, loading }. Simule o login com email/senha (mock).
2. Use React Router v6: defina rotas /, /login, /dashboard, /profile. Proteja /dashboard e /profile: redirecione para /login se não autenticado.
3. Implemente PrivateRoute como componente wrapper que verifica o AuthContext e renderiza <Outlet /> ou <Navigate to="/login" />.
4. Pesquise JWT (JSON Web Token): header.payload.signature. Como armazenar tokens no front-end (localStorage vs cookies httponly)? Quais os riscos de XSS?

Exercício 3

Cenário: Dashboard financeiro que busca dados de múltiplas APIs e gerencia cache.

1. Crie hooks customizados: `useFetch(url)` que retorna `{ data, loading, error }`, `useLocalStorage(key, initial)` para cache.
2. Implemente um dashboard com: saldo atual (API A), últimas transações (API B), gráfico de gastos por categoria (Chart.js ou Recharts).
3. Adicione cache com `stale-while-revalidate`: mostre dados em cache primeiro, atualize em background quando a API responder.
4. Explique o problema de race condition em requisições assíncronas: como garantir que o resultado de uma requisição anterior não sobrescreva o de uma mais recente? (`useEffect cleanup`, `AbortController`).

Exercício 4

Cenário: Formulário de cadastro de produtos com upload de imagem e preview.

1. Crie um formulário com: nome, descrição (textarea), preço, categoria (select), imagem (input type=file).
2. Implemente preview da imagem antes do upload: use `FileReader` ou `URL.createObjectURL()`.
3. Simule o envio do formulário para uma API (POST `/api/products`) com `FormData` contendo os campos e o arquivo.
4. Adicione barra de progresso no upload usando `XMLHttpRequest.upload.onprogress` ou `fetch` com streaming.

Requisitos

Node.js 18+, Vite + React. API Key OpenWeatherMap (gratuita) ou alternativa. Chart.js ou Recharts.